

SpeedyMeals — Product Architecture

What are we building?

Not just a food app. A future ecosystem — food delivery, ride booking, medicine delivery, technician booking, logistics — all under one platform, working in multiple countries eventually. But we start small (food delivery only) and grow module by module.

Big Decision #1: How is the system organized?

We are using something called a "Modular Monolith."

Simple meaning: It's ONE single app (one backend, one deployment) — but inside, it's cleanly divided into separate sections, like separate rooms in a house. Each "room" (module) handles one business — food, ride, medicine, etc. — and doesn't mix with the others.

Why not build it as 5 totally separate apps (microservices) since we plan 5 businesses?

Because right now we have zero users. Building 5 separate systems today means 5x the cost, 5x the complexity, and a team big enough to manage all of it — none of which we need yet. It would be like building 5 separate shops before we've sold a single sandwich.

Why not just throw everything into one messy pile of code either?

Because that becomes impossible to manage once we add more modules — bugs everywhere, no one understands anything, adding ride-hailing later becomes a nightmare.

So the middle path: one app, cleanly divided rooms. Cheap and simple now, but each "room" can be pulled out into its own full system later without rebuilding everything — because we built the walls properly from day one.

Big Decision #2: What are the "rooms" (modules)?

Two types:

Shared Foundation (used by everything):

- Login/Accounts (Auth)
- User Profiles
- Wallet & Payments
- Notifications
- Location/Maps

Business Modules (the actual industries, added one at a time):

- Food Delivery ← we build this first (MVP)
- Ride Hailing ← later
- Logistics ← later
- Medicine Delivery ← later
- Technician Booking ← later

Why separate this way: Login and Wallet work the same no matter if someone's paying for food or a ride. So we build that ONE time, and every future module reuses it. Saves time, avoids duplicate bugs.

Big Decision #3: The Golden Rule

No module is allowed to directly reach into another module's data. If Food module needs money moved, it has to "ask" the Wallet module properly — not go grab the data itself.

Why this matters: This is the rule that makes our future growth painless. If Ride-Hailing becomes huge one day and needs to become its own separate system, we can pull it out cleanly — because it never depended on anyone else's internals in the first place.

Big Decision #4: One Shared Database (for now)

All modules currently share one database, just clearly labeled by module (like labeled drawers in the same cabinet).

Why: Running 5 separate databases today is unnecessary cost and complexity with no real traffic yet. But we ARE preparing the data now to support multiple countries (adding country and currency info to every relevant record from day one) — cheap to add now, very expensive to fix later once we have real data and are expanding to Saudi, UAE, Europe, etc.

Big Decision #5: One Codebase, Three Apps Inside

One project folder holds three things:

- Backend (the engine/API)
- Admin Web Panel (for managing the business)
- Mobile App (for customers/riders/technicians)

Why: At our current team size, everyone benefits from seeing the whole system in one place instead of hunting across scattered repositories. Easier to review, easier to keep everything in sync.

Big Decision #6: We Show the Full Vision From Day One

Even though we're only building Food Delivery right now, empty placeholders exist for Ride, Logistics, Medicine, and Technician from the very start.

Why: Anyone reviewing the project — technical or not — immediately sees the real, intended shape of the full ecosystem, not just "a food app." It also means when we're ready to build the next module, there's already an agreed, clean place for it.

What we are deliberately NOT building yet

Separate systems per module, multi-country servers, complex messaging systems, heavy background job processors, push notifications, advanced monitoring tools.

Why: These solve problems we don't have yet (huge traffic, multiple big teams, complex workflows). Building them now would slow us down solving imaginary problems instead of shipping the real MVP.

In one sentence:

We're building one simple, well-organized system today that's structured so it can honestly grow into the big multi-service ecosystem we're imagining — without ever needing to throw away and rebuild the foundation.

Folder Structure





